競技部門のルール

今年の競技部門では、はじめに「マップ」と呼ばれる盤面の情報が与えられます。マップは六角形の「セル」によって構成され、マップ内にはうどんを獲得できる「スポット」が点在しています。プレイヤーは２種類のエージェントの経路を決めて、マップ内を移動させることができます。経路を決めるには目的地までの最短経路だけではなく、エージェントが移動で消費する燃料も考慮する必要があります。効率よくスポットを巡回し、一番多くうどんを獲得したチームが勝利です。

日数と地形タイプ

指示を与える最小時間単位を「ステップ」と言います。この競技では試合の進行を複数の「日」に区切って構成し、１日の活動時間は日ごとにステップ数で定めます 。1試合は４～10日で行われます。

マップを構成するセルには地形タイプが割り当てられています。地形タイプは「平地」、「山地」、「池」、「道路」の４種類があります。平地・山地・道路には移動することができますが、移動にかかるステップ数が異なります（表1）。さらに、道路は「順調」、「混雑」、「渋滞」の３種類の状態があり、前日と前々日の２日間の「交通量」によって状態が決まります。池には移動することができません。

エージェント

各チームは複数のエージェントを管理します。エージェントには「巡回車」と「補給車」の２種類があります。巡回車はうどんを獲得できますが、燃料が足りない場合は移動できません。補給車はうどんを獲得できませんが、巡回車と同じセルに移動することで巡回車に燃料を補給することができます。また、補給車は移動に燃料を必要としません。

スポット

マップ内のいくつかの平地セルにはスポットが配置されています。巡回車がスポットに到着すると、自動的にうどんを獲得することができます。１巡回車が１スポットから１日に獲得できるうどんの数は、１玉までです。スポットはいくつかの「系列」に分かれていて、系列ごとにうどんの「種類」が異なります。

順位決定方法

以下の優先順位で勝敗を決定します。

(1) １試合で獲得した**うどんの種類数**が多いチームを勝者とします。

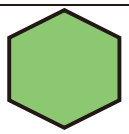
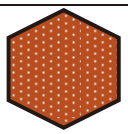
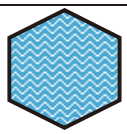
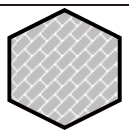
(2) 各日に獲得した**うどんの種類数の累積**（日ごとの種類数の合計）が多いチームを勝者とします。

(3) １試合で獲得した**うどんの玉数の総和**が多いチームを勝者とします。

(4) 各日の最後に送信した**有効な回答の回答時間**の累積が少ないチームを勝者とします。

(5) サイコロで勝敗を決めるか引き分けとします。

表１．セルの地形タイプ

イメージ						
地形タイプ	平地	山地	池	道路		
移動時間（ステップ）	2	3	進入不可	1（順調）	2（混雑）	4（渋滞）
燃料の消費量	1	2	−	2		

競技部門の組合せ

1回戦

ブース	第1試合	第2試合	第3試合	第4試合	第5試合	第6試合
1	佐世保	群馬	一関	富山（本郷）	徳山	神戸市立
2	大分	香川（高松）	鈴鹿	都城	阿南	鳥羽商船
3	石川	苫小牧	奈良	都立（品川）	東京	仙台（広瀬）
4	大阪公大	弓削商船	鶴岡	サレジオ	久留米	和歌山
5	神山	長野	広島商船	沼津	釧路	津山
6	豊田	熊本（八代）	宇部	仙台（名取）	熊本（熊本）	小山
7	舞鶴	旭川	木更津	明石	富山（射水）	大島商船
8	秋田	鹿児島	新居浜	米子	沖縄	香川（詫間）
9	松江	福井	有明	茨城	八戸	長岡
10	呉	近畿大	ハノイ国家大	新モンゴル	モンゴル科技大	

※各試合上位5チームが準決勝へ進出します。6位以下は敗者復活戦への出場が決定します。

敗者復活戦

ブース	第1試合	第2試合	第3試合
1	1－1－6	1－3－6	1－5－6
2	1－2－6	1－4－6	1－6－6
3	1－3－7	1－5－7	1－1－7
4	1－4－7	1－6－7	1－2－7
5	1－5－8	1－1－8	1－3－8
6	1－6－8	1－2－8	1－4－8
7	1－1－9	1－3－9	1－5－9
8	1－2－9	1－4－9	1－6－9
9	1－3－10	1－5－10	1－1－10
10	1－4－10		1－2－10

※各試合上位2チームが準決勝へ進出します。

準決勝

ブース	第1試合	第2試合	第3試合	第4試合
1	1－1－1	1－2－1	1－3－1	1－4－1
2	1－5－1	1－6－1	1－6－2	1－5－2
3	1－4－2	1－3－2	1－2－2	1－1－2
4	1－3－3	1－4－3	1－5－3	1－6－3
5	1－2－3	1－1－3	1－1－4	1－2－4
6	1－6－4	1－5－4	1－4－4	1－3－4
7	1－1－5	1－2－5	1－3－5	1－4－5
8	1－5－5	1－6－5	C－3－1	C－2－2
9	C－1－1	C－2－1	C－1－2	C－3－2

※各試合上位2チームが決勝へ進出する。
※上位2チームとは別に、準決勝上位チームの中から別途定める方法により決勝進出校を2校決定します。

決勝戦

ブース	第1試合
1	S－1－1
2	S－2－1
3	S－3－1
4	S－4－1
5	S－1－2
6	S－2－2
7	S－3－2
8	S－4－2
9	L－1－1
10	L－1－2

※X-Y-Z　は X 回戦-第 Y 試合-第 Z 位　を表します。
※ただし、X 部分のCは敗者復活戦、Sは準決勝、Lは別途定める方法で出場校を決定します。

・提出された原稿をそのまま印刷しています。

1 おぉ

仙台
(広瀬)

工藤 聖晴（5年） 梅津 遥真（2年）
渡邉 俊介（2年） 奥村 俊昭（教員）

1. 概要

本競技では、巡回車と補給車を合流させて同時に動かすことで、燃料無限の巡回車として扱うことができる。これにより、複雑な補給タイミングの計算を排除し、純粋な経路選択の問題へと簡略化する。

次に、Dijkstra 法で各エージェント、各スポットから自分以外の各スポットへの最短距離を求める。各エージェントにスポットを暫定的に割り当てて個別の経路を構築し、これを初期解とする。

この初期解をもとに、焼きなまし法を用いて、最終的な巡回経路を探索・最適化する。

2. 他チームへの対策

渋滞を故意に発生させる（例：全てのエージェントを補給車にし、ひたすら道路を巡回して妨害する）チームが現れるかもしれない。そのため、道路に依存しない初期解も作成し、ビジュアライザ等を確認しながら複数の巡回経路を切り替えて解く。

3. 開発環境

言語：C++、Python、HTML、CSS、JavaScript

エディタ：Visual Studio Code

AI・開発支援：Gemini



図. 使用するビジュアライザ

2 ゲシュタルト崩壊、うっっっ これはベンゼン環！？

舞鶴

塚本 樹（4年） 中島 拓海（3年）
塩見 享大（3年） 森 健太郎（教員）

1. 概要

各スポット間の最短経路をダイクストラ法で求め，その結果を基に焼きなまし法を用いて巡回順序を探索する．よりスコアの高い巡回経路を探索し，解答として採用する．

2. アルゴリズム

2.1 ダイクストラ法

地形ごとの移動時間や燃料消費量を移動コストとして数値化し，ダイクストラ法を用いて各スポット間の移動コストが最小となる経路を求める．

2.2 焼きなまし法

ダイクストラ法で求めたスポット間の移動コストを利用し，獲得できるうどんの種類数や総数などを基に経路を評価する．スポットの巡回順序を繰り返し変更しながら探索することで，より効率の良い巡回経路を求める．

3. ビジュアライザ

盤面の状況を直感的に把握するため，六角グリッド上に地形，スポット，エージェントを表示するビジュアライザを実装した．また，各エージェントの移動履歴を確認できるようにした．



4. 開発環境

エディタ：Visual Studio Code，Visual Studio 2019

言語：C++14，Python3

3 おお

八戸
長谷川 隼也（5年） 金田 空悟（4年）
小山田 零風（2年） 細川 靖（教員）

1. アルゴリズム

1.1 初期解

1 つのスポットから回収できるうどんの最大量は min(在庫, 巡回車数) で表せる。よって各スポットにその台数分の担当を割り当て、訪問順を最近傍法で並べる。スポット間の最短経路は Dijkstra 法で求める。

1.2 焼きなまし法

前述のような貪欲法では局所解に陥るため、初期解に対して焼きなまし法を適用した。巡回車の担当スポットの追加や、2-opt など多様な近傍操作により局所解を脱し、より多くのうどん獲得を目指した。

1.3 評価関数

勝利条件は、種類数➡種類数の累積➡玉数の総和である。そこで、上位の項が下位を必ず上回るよう桁を分けることで優先順位を表現する評価関数を設計した。

2. ビジュアライザ

下図に示すビジュアライザを作成した。燃料・獲得系列・うどんの数の表示に加えて、軌跡・再生速度・巻き戻し・マップ自動生成など多くの機能を備えている。実際に複数プレイヤーで試合を行うことができ、効率的なソルバ改善を行うことができる。

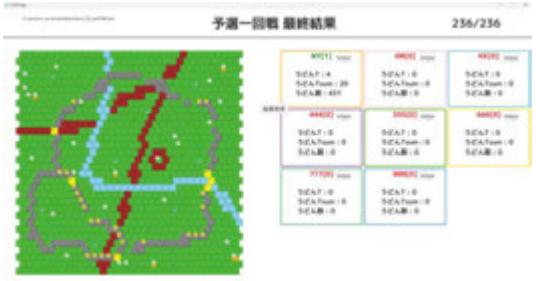


図 ビジュアライザ

3. 開発環境

言語 C++/Python その他 OpenMP/Siv3D

4 T-Seto Nexus

津山
杉本 昊大（4年） 岩田 紘生（2年）
横浦 佑亮（1年） 宮下 卓也（教員）

1. アルゴリズム

幅優先探索(BFS)を行い、スポットを巡る。一日目は全道路が「順調」のため、道路を積極的に通り、妨害とうどんの獲得を目指す。二日目以降は平地が多い経路と道路(順調)が多い経路を比較して、累計ステップ数が低い方を通る。全種類のうどんを集めることを最優先に、うどんの玉数を増やすことを次の目標として動く。

2. 工夫点

マップ全体の道路の 80%以上の状態が「渋滞」の場合は道路と平地の比較ではなく、山地と平地での比較を用いる。巡回車は、基本的に補給車から離れないように動かす。また、万が一に備えて巡回車は燃料の 1/2 以上を各スポットにつくまで残しつつ行動する。進行中はなるべく道路(順調)を通り、簡易的にでも妨害を行っていく。

3. GUI

Siv3D を使ってビジュアライザーを作り、視覚的にわかりやすくしている。

4. 開発環境

【OS】Windows 11 home
【Language】C++
【Frame work】Siv3D
【IDE】Visual Studio
Visual Studio Code

5 อุดรธานี

東京

豆田 匠音（4 年） 吉田 宗司（4 年）
石井 岳光（4 年） 松崎 頼人（教員）

1. 解法について

私たちは本問題を、燃料制約・補給車との合流を伴う「拡張版の巡回セールスマン問題」として定式化した。各日の巡回車の訪問順序を焼きなまし法で最適化し、経路の組み替えやスポットの入れ替えといった近傍をランダムに試しながら、悪化する解も一定確率で受け入れることで局所解に陥ることを避けている。評価関数には燃料切れや道路の混雑といった要素も組み込み、採点機と同じ規則でシミュレーションしながらスコアを計算する。初期解として貪欲法を用いて大まかな骨格を作り、そこから焼きなまし法で最適化するという二段構えの構成である。

2. 開発したツール

ブラウザで動くビジュアライザを作成した。マップを表示し、エージェントの軌跡やスコアの推移を再生できるようにした。焼きなまし法がどのような巡回計画に辿り着いたかを目で確認し、ソルバの挙動を分析・デバッグするために使用している。

3. 開発環境

- OS: macOS / NixOS / Windows11
- エディタ: VSCode, NeoVim
- 使用言語: C++, Rust, Python, TypeScript / Svelte



図 1 作成した Visualizer

6 焼きなましうどん ～ネストマシマシ 計算量マシ コード固め～

大分

川並 裕斗（3 年） 池田 宗志郎（2 年）
斎藤 駿喜（2 年） 徳尾 健司（教員）

1. 概要

今回の競技では，個々の戦術を実行した際の試合結果をデータベース化し，最適な戦術を自動的に選ぶプログラムを用意した．試合情報を受け取るとその状況に最も適している戦術が選択され実行される．

2. 競技中の流れ

競技開始とともに試合情報を受け取り，事前に用意しておいた複数の戦術のうち，どの戦術を使用するかを上記のデータベースを参照し選び出す．その後，戦術を用いて日ごとに移動計画を決定する．決定した移動計画は自動的にサーバーに送信される．

3. 戦術内で用いる手法

選択した戦術内では焼きなまし法と評価関数という 2 つの手法を主に使い分ける．焼きなまし法はスポットの訪問順序を最適化する手法であり，評価関数は個々のエージェント周囲のマスを得点付けして比較する手法である．

戦術では，これらの手法を単独で使う戦術の他に，2 つを併用し，連携させる戦術も用意した．焼きなまし法単体では順序の最適化以外の部分が不得意であり，反対に評価関数単体では順序の最適化の部分が不得意である．したがって，この 2 つの手法を組み合わせることにより，互いの不得手を補い合うことができる．この工夫により，「2 つの手法の組み合わせ戦術」では，現実的な計算時間内で，最適解により近しい移動計画を導出させることが可能である．

4. 開発環境

IDE: Visual Studio Code, Visual Studio 2022
OS: Windows 11 Home, Ubuntu 24.04 LTS
開発言語: C++, Python
ライブラリ: Siv3D

7 最高速度で回収したる

熊本（八代）
橋口 直弘（3年） 三重 諒真（3年）
岩﨑 裕生（3年） 小島 俊輔（教員）

1. はじめに

補給車が巡回車に合わせる形で探索を行い，より多くのうどんを回収することを目指すプログラムを開発した．

2. 解法アルゴリズム

はじめに，一台ずつおおまかな巡回車の経路を生成し，補給車を巡回車の経路に合わせて割り当てる．その後局所的な改善を繰り返して最適解を出力する．

また，今回の競技では燃料補給と渋滞の回避が重要であるため，巡回車の探索範囲を渋滞の影響が少なくかつ価値の高いところのみに絞り安定化を図る．ただし，この方法では種類数が増やせなくなる恐れがあるため，範囲を絞り安定回収するエージェントと種類数を優先して遠征するエージェントに役割分担をする形で探索を行う．

3. ビジュアライザについて

エージェントの動きやうどんの回収状況を直観的に理解するため OpenSiv3D を使用して図 1 のビジュアライザを作成した．ビジュアライザを用いることによりバグの発見，アルゴリズムの改善を効率的に行うことが可能となる．

4. 開発環境

言語：C++, Python, TypeScript, C#

使用ライブラリ：OpenSiv3D



図 1：ビジュアライザ

8 U don't know

石川
柳梅 舶斗（3年） 門廻 亨弥（3年）
濵田 竜進（3年） 越野 亮（教員）

1. 概要

スポットをクラスタにまとめることで探索を効率化し、うどんの種類数を増やしながら移動コストを抑えた。また、GUI を用いて探索の一部を手動で行うことで、より効率的な探索を実現した。

2. 解法

2.1 クラスタ作成

Web 上で動作する図 1 の GUI を用いてスポットを選択し、クラスタを作成する。作成したクラスタについて、ダイクストラ法などによってクラスタ内の巡回コストや、クラスタ間の移動コストを見積もる。

2.2 巡回経路の決め方

各クラスタの巡回コストを基に、各巡回車が担当するクラスタを決める。各案について、獲得できるうどんの種類数を優先して評価し、クラスタの割り当てや巡回順序を入れ替える局所探索によって経路を改善する。

また、巡回車の燃料が少なくなる地点を予測し、補給車が同じセルで合流できる地点を探索して、補給車の経路を決める。

3. GUI について

図 1 の GUI では以下の操作が可能である。

・マップ表示
・クラスタ作成
・クラスタ巡回順序の選択
・探索経路の表示



図 1 経路探索 GUI

9 淵底

明石

笠井 瑛司（5年） 上　天将（4年）
布谷 太一（3年） 野村 隼人（教員）

1. アルゴリズム

本問題は巡回セールスマン問題と問題設定が類似していることから、焼きなまし法を採用した。具体的には、各日の開始時に各スポット間の最短距離をダイクストラ法で計算し、未回収の系列を優先して回収する貪欲法で初期解を作成する。その後、複数の近傍を用いて焼きなまし、解の改善を行う。盤面の評価には、獲得系列数、1日ごとの獲得系列数の合計、獲得玉数を辞書順に重み付けした評価関数を用いる。

2. テスタ

ソルバの定量的な評価を行うため、一人用、対戦用の2種類のテスタを開発した。対戦用テスタでは、複数回対戦させ勝率の比較等が可能である。また、エージェントの経路を視覚化するためビジュアライザの開発も行った。これにより人力での出力の確認を容易にする。



図1 ビジュアライザの表示例

3. 開発環境

エディタ: Visual Studio Code

言語: C++ / Rust

10 田舎村 v2

茨城

井村 元紀（3年） 中村 充希（3年）
川村 蒼太朗（3年） 安細　勉（教員）

1. 概要

勝敗判定で優先される「1試合で獲得したうどんの種類数」と「各日に獲得したうどんの種類数の累積」を重視し、これらをできるだけ大きくするよう巡回車と補給車の行動を決定する。

2. アルゴリズム

2.1 スポットの割り当て

マップ全体を考慮し、巡回経路への追加しやすさや系列の担当しやすさから、各巡回車が担当するスポット・系列を決定する。

2.2 移動経路の決定

地形タイプや道路の状態を考慮し、ダイクストラ法で地点間の移動コストを求め、限られたステップ数で効率よくスポットを巡る経路を構築する。

2.3 局所探索による改善

局所探索で巡回順や担当を改善し、燃料と補給車による補給を考慮して実行可能性を確認する。

3. ビジュアライザー



図1 マス

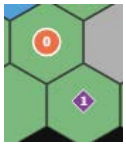


図2 スポットとエージェント

巡回車と補給車の行動を視覚的に確認するため、ビジュアライザーを作成した。エージェントの移動やスポットの位置・系列・在庫などを確認できる。

4. 開発環境

使用言語：C++ / Python

ビジュアライザー：HTML / CSS / JavaScript

11 四国無双十三麺待ち

佐世保
内海 陽太（3年） 廣重 伊織（5年）
林田 瑞樹（3年） 嶋田 英樹（教員）

1. 概要

試合で有効となる回答は最後に受理された回答であることと，勝敗判定に優先順位があることを考慮して，２つのアルゴリズムを用いて競技を行う．

2. アルゴリズムについて

競技では動かすエンティティを一つ選び，目的地への最短経路を導くアルゴリズムを用いて回答を行う．

2.1 操作するエンティティを選択するアルゴリズム

まだ取得していない種類のスポットに一番近い巡回車を操作し、全種類集めきった後は各巡回車から目的地までの距離が近いものから操作していく。

補給車はマップ中に点在させ、基本は動かさないものとする。

2.2 最短経路で目的地まで移動するアルゴリズム

このアルゴリズムは一つのエンティティに着目し、巡回車をスポットへ、補給車を巡回車へ最短経路で移動させることを目的としたアルゴリズムである。この目的の実現のためにA＊アルゴリズムを適用する．

A＊アルゴリズムは与えられたスタート地点からゴール地点までの最適なルートを探索するためのアルゴリズムである。エンティティの初期地点から現地点までの必要ステップと、現地点から目的地までの距離によって決まる予測コストを設定し、コストが最小となるルートを探索することにより最短ステップで進めるルートを求める。

[OS] Windows11
[言語] Visual C++
[IDE] Visual_Studio

12 沖縄そば

沖縄
當山 浩生（4年） 上原 尚（2年）
具志 遙弥（4年） 金城 篤史（教員）

1. 概要

巡回車はスポットを巡回しながら、多様な種類のうどんをできるだけ多く獲得するとともに、補給車との合流も計画する必要がある。本システムでは、まず巡回車の経路候補を生成し、次にチーム全体の組合せ探索を行い、最後に補給車の経路を決定することで問題を解決する。

2 巡回経路候補の生成

スポット間の最短経路を事前計算し、各巡回車の訪問順候補を複数生成する。その際、候補が類似した経路に集中しないよう、取得できる系列の組合せと終了位置の多様性を考慮する。

3 チーム全体の組み合わせ探索

各巡回車の経路候補を分枝限定法で組み合わせたチーム全体を、系列数、日別系列数、獲得玉数の順に評価する。探索中は、現在の最良解を上回らない組合せを枝刈りし、計算量を削減する。

4 補給車の経路決定

3 で求めた巡回車の行動計画から補給要求を抽出し、補給時刻および補給地点の候補を生成する。次に、補給が不可能な候補を事前に除外した上で、補給要求の実施状況を状態として管理する動的計画法（DP）により、補給計画の実現可能性を判定する。実現可能な場合は、DP の遷移をたどることで補給車の行動計画を復元する。

巡回候補生成 → 候補選別 → 分枝限定 → 補給 DP → 行動出力

図 1　解答生成の流れ

5 OpenSiv3D によるデバッグ

解答は基本的に自動生成するが、巡回車および補給車の行動計画の把握、複数の解答候補の比較、アルゴリズムのデバッグを容易にするため、OpenSiv3D を用いて計画を可視化する。

4. 開発環境

使用言語：C++
ライブラリ：OpenSiv3D 0.6.16

13 O (UDON)

一関

吉川 礼人（4年） 菅原 健心（3年）
三留 駿（3年） 小林 健一（教員）

1. はじめに

勝敗では、獲得したうどんの種類数が特に重視されるため、本システムでは未獲得の種類を優先して回収することを基本方針とした。

また、巡回車の燃料制限や補給車との合流、日ごとに変化する道路状態を考慮し、毎日安定して有効な行動計画を作成する。

2. アルゴリズム

盤面を六角形セルのグラフとして扱い、Dijkstra 法で重要地点間の経路を求める。さらに、うどんスポットなどの重要地点だけを用いた圧縮グラフを作成し、訪問順序を高速に評価する。

まず貪欲法で初期解を作成し、その後、焼きなまし法で訪問順序や巡回車間の担当を変更して解を改善する。巡回車の経路はセル単位・時刻単位に展開し、補給車との合流可否を確認することで、燃料切れを防ぐ。

3. 特徴

本システムでは、当日の獲得数だけでなく、翌日の開始位置も評価に含める。日末に重要スポットの近くや補給しやすい位置で終了する候補を高く評価し、次の日の初動を有利にする。

また、複数台の PC を用いて候補評価を分担する。メイン PC は候補生成と回答作成を行い、別 PC で翌日評価や補給成立性の検証を行う。補助 PC の結果が間に合わない場合でも、メイン PC 単独で有効な回答を作成できる。

4. 開発環境

使用言語：C++, Python

開発環境：WSL2 Ubuntu, Visual Studio Code

14 本郷パーティ

富山（本郷）

石王丸 豊年（3年） 永言 颯汰（3年）
西村 旭生（専攻科1年）
石田 文彦（教員）

1. アルゴリズムについて

本システムでは，六角格子状のマップ上で A*アルゴリズムを用いた経路探索を行い，巡回車と補給車の行動を決定する。探索では格子上の移動距離を基準とし，「地形による移動コスト」「燃料消費」「道路状態」などを加味して経路を評価する。

巡回車は，すべてのうどんスポットを候補ゴールとして A*探索を行い，総コストが最も低い地点を目的地として選択する。

補給車も同じ探索アルゴリズムを利用するが，ゴールを巡回車の位置に設定する点が異なる。また，燃料残量の少ない巡回車ほど優先して補給対象となるよう重み付けを行うことで，安定した運用を実現する。

2. 本アルゴリズムの独自性について

本システムの独創的な点は 2 つある。第一に，巡回車の燃料が一定値以下になると，補給車がその車両を優先的に支援するよう重み付けを行い，燃料不足に自動で対応できる。第二に，「道路状態」「地形」「うどん在庫数」「交通量」などを別々に管理しつつ同じ座標に対応づけることで，1 つのマスから複数の情報を取得でき，「混雑回避」「燃料効率」「うどん獲得の可否」を総合的に考慮した経路探索を実現している。

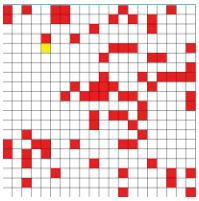


図 1 実装中の A*アルゴリズムマップ

15 苫[CENSORED][CENSORED]高専

苫小牧

野沢 和真（4年） 山内 颯大（4年）
鍋嶋 謙太（3年） 三上 剛（教員）

1. 概要

スポットや燃料不足の巡回車を巡る順番を選択し，ルートを生成する際には，以下2節で挙げる手法を使用する．これらの手法を同時に使用することで，最適な解を得る．

また，二つの手法それぞれにおいて，並列処理を行うことで，求解時間の削減を目指す．

2. 解の探索

2.1. ダイクストラ法

第一の手法は，ダイクストラ法を用いてルートを生成する手法である．

いくつかのスポットをまとめ，スポットのグループを構築する．そのスポットのグループには一台の巡回車が割り当てられ，そのグループ内でスポットを巡回する．

補給車は各グループの巡回車を回り，巡回車に燃料を補給する．

2.2. 線形計画法

第二の手法は，線形計画法を用いて各スポットと燃料を補給する巡回車を選択する手法である．

巡回車が各スポットを巡り，一日に取得するうどんの系列数を最大化するようにスポットを選択する．また，巡回車は燃料切れにならないようにルートを選択する．

補給車については，燃料切れを起こしそうな巡回車を優先して回ることにする．

3. 開発環境

OS：Ubuntu 24.04 (LTS)

開発言語：C++

使用ライブラリ：cpp-httplib, nlohmann json, HiGHS

16 TeCうどん

徳山

木村 遼太（2年） 藤井 煌太（2年）
米谷 颯二郎（1年） 力 規晃（教員）

1. 概要

スポットを巡る経路は多くのパターンが考えられ、計算量を減らす必要がある。そこで、ダイクストラ法とビームサーチを用いる解法を採用する。

2. 解法

まず 1 対 1 のスポット間の最短経路をダイクストラ法ですべて求める（図 1）。次に、スポットを短い道のりでかつ得点が高くなるように巡回する経路をビームサーチで求める（図 2）。経路を求めつつ、巡回車の動きをシミュレートし、在庫がなくなったスポットは通らないように経路を調整する。

また補給車については、巡回車の動きをシミュレートした際に、燃料が切れる順に巡回するような経路を求める。

3. 開発環境

OS: Windows 11 Home、 Debian 13、 Ubuntu 24。04

プログラミング言語: C++17

外部ライブラリ: libcurl、 nlohmann-json

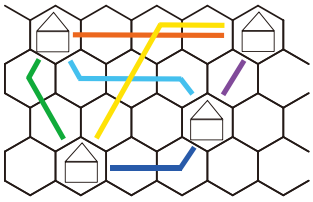


図 1 すべてのスポット間の経路

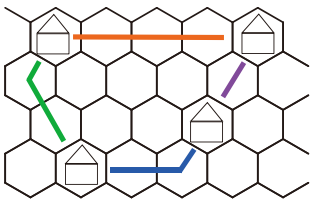


図 2 ビームサーチで求めた経路

17 うどんとりものがたり

香川（詫間）

野田 八雲（3年） 小山 昊想（1年）
和氣 典汰（3年）
TIN HTAY HLAING（教員）

1. システム概要

本システムは、与えられたマップから最適な回り方を求めるアルゴリズム部とアルゴリズムの求めた移動経路を確認できるビジュアライザ部からなる。

2. 高速化について

渡されたマップをいくつかのマップに分けて、最適な回り方を探索し、そのマップの上に巡回車を配置させることでマップ全体を見ずに探索を行うことができる。

3. アルゴリズムについて

分けたマップに山登り法や、A*アルゴリズムを使用することで最短距離を計算し、そこの道を回し続けるという方法をとる。

4. ビジュアライザ

ステップごとのエージェントの位置や今までのログの確認が行え、ボタンによるアルゴリズムの変更、回答の提出を行えるビジュアライザを作成した。

5. 開発環境

言語:C++ ,Rust ,Python

ライブラリ:OpenSiv3D



図　ビジュアライザ予定

18 エクストラエクストリーム エクスプレスエクスペリメント

有明

古賀 雅寛（4年） 竹内　舜（4年）
畑田 ひろ（4年） 森山 英明（教員）

1. システムの概要

本システムでは、ゲーム内でより多くのうどんを獲得するための移動計画を決定し、出力する。これを実現するため、最適経路を算出するアルゴリズム（後述）を用いて、また 1 ステップごとに残り燃料や位置を事前に確認する処理によって車が途中で動けなくなるのを防ぎながら無駄のない行動計画を探索する。

2. 巡回車の経路を決定するアルゴリズム

効率よくスポットを巡るために、マップの全スポット間における最短移動手順を算出する方法を採用する。このとき、道路の渋滞状況に応じたコスト計算を工夫することで、効率よく目的地まで巡回する。最適経路探索と方向コードによる移動例を図 1 に示す。図 1 では出発地点を起点にして、複数のスポットを巡回する経路を示している。この最適な巡回ルートは以下の手順で決定することができる。

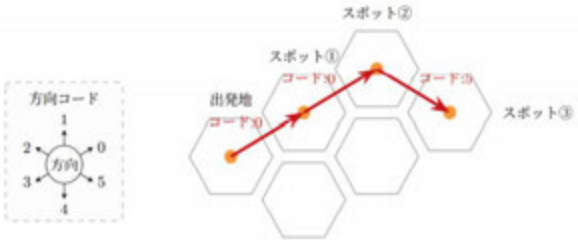


図 1：最適経路探索と方向コードによる移動例

Ⅰ）各スポット間の最小コスト（移動にかかる時間や燃料）をダイクストラ法により算出し、全地点間の移動コストをまとめた距離行列を構築する。Ⅱ）得られた距離行列をもとに、最も効率的な訪問順番を導き出す Held-Karp アルゴリズムを実行し、最適な巡回順序を決定する。Ⅲ）移動可能なら図 1（方向コードの凡例）に従い、移動先の方向（0～5）を指定して移動を行う。移動不可なら待機指示を行動計画に追加する。

19 ICT 帰宅部

呉
森田 光博（3 年） 河村 創太（2 年）
白井 陽士（3 年） 城明 舜磨（教員）

1. アルゴリズム

1.1 経路探索

六角格子上の各セルを頂点、隣接セルへの移動を辺としてグラフ化し、地形・道路状態に応じた移動ステップ数を辺コストとしてダイクストラ法により最短経路を求める。道路セルは順調・混雑・渋滞の 3 状態を持ち、状態によって必要ステップ数が変化するため、各日開始時に与えられる道路状態を反映したグラフを構築する。さらに、池セルは侵入不可であるため、探索時点で候補から除外する。

本問題のマップサイズは最大でも 32×32 程度であり、到達可能セル数は高々約 1000、辺数も約 6000 程度であるため、ダイクストラ法を用いた探索は十分高速に実行可能である。したがって、各日・各エージェント・各候補スポットに対して複数回探索を行い、最適な移動先を決定する構成とする。

1.2 行動決定

実際の行動決定にはビームサーチを用いて行動列全体を探索する。ビームサーチでは、各エージェントの現在位置・残り燃料・残りステップ数・取得済み系列・スポット在庫・補給車位置などを状態として保持し、候補行動を段階的に展開する。探索時には、「移動」「待機」「補給車との合流」など複数の行動候補を生成し、将来の獲得可能性を考慮しながら有望な状態のみを残すことで、探索範囲の爆発を抑制する。

2. ビジュアライザー

Siv3D を用いたビジュアライザーを作成し、エージェントの移動・補給・渋滞変化を可視化することでアルゴリズムの検証と改善を行う。

3. 開発環境

言語 C++ フレームワーク Siv3D

20 le meilleur

広島商船
関野 翔太（5 年） 吉山 幸志（5 年）
中岡 大誠（5 年） 岩切 裕哉（教員）

1. はじめに

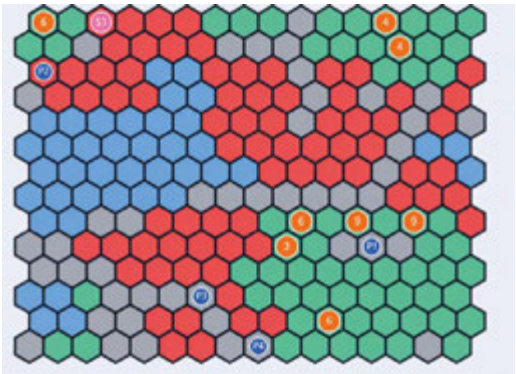
今回の競技部門では、エージェントを操作し、マップ上のスポットをできるだけ巡り、うどんを集めていくというものである。 勝敗判定は 1. うどんの種類数 2. 種類数の累積 3. うどんの玉数の総和 4. 解答時間の累積 と種類が上位を占めているため、毎ステップ効果的にうどんを集めることに注力する。

2. 戦略

セルごとの消費ステップ数と消費燃料数のどちらもを経路探索のアルゴリズムに組み込むことで、よりエージェント達にとって負荷の少ない経路を導き出せるようにしている。

また、同じスポットへ行こうとしている場合はどちらが行った方がより効率的なのか比較する処理もしている。

効率を求める全ての評価関数は試行を何度も繰り返すことで求める。



3. 開発環境

OS ： macOS Tahoe 26.1,
Windows 11
使用言語：python
エディタ ： Visual Studio Code

21 IOS.tar.gz.b64

木更津
諏訪　翼（3年）　石田 伊織（3年）
大阿久 怜志（3年）　大枝 真一（教員）

1. 概要

各日のはじめに、全頂点間同士のステップ数をダイクストラ法で算出しておく。マップサイズの制約からこれは十分高速に動作する。これをすることで、それぞれの頂点から最も近いスポットまでのステップ数がわかる。 そのため、巡回車を貪欲法、補給車をビームサーチで探索する。

2. アルゴリズム

2.1 ダイクストラ法

マップの頂点の数を N とすれば、計算量 N^2 で動作する。始点と終点のペア間のステップ数を保持しておく。巡回車にこの情報を用いて無駄な計算を減らし、高速化に充てる。

2.2 ビームサーチ

補給車における探索として、未取得うどんブランド、燃料、残りステップ、巡回車が次に向かっているスポットを都度評価し最適な巡回経路をビームサーチで探索する予定である。

3. ビジュアライザ

Go 言語によって実装している。巡回車や補給車の行動、燃料消費、待機時間などを可視化・統計処理することで、戦略の改善に役立てる。

4. 開発環境

言語 C/C++

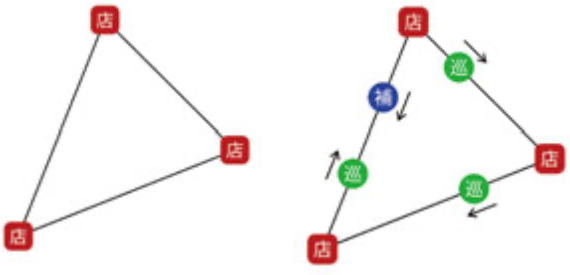
22 鬱鬱鬱

宇部
山本 健一朗（5年）　池本 悠生（4年）
木村 友亮（4年）　田辺　誠（教員）

戦略

まず, フィールド全体のうどん屋を巡回する大きなループを作成し (図 a). 試合が開始したら各エージェントをそのループ上に配置し, その後は巡回車は右回り, 補給車は左回りにループを回るようにする (図 b). このようにすることで, 巡回車が 1 周する間に補給車の数の 2 倍の回数の燃料補給を行うことができる.

最初のループの設定は燃料消費量と移動時間, 最初の移動にかかる時間や, 道路の混雑さの不確実性などを考慮して, よりよいものを探索する.



(a)　(b)

環境

3 台の PC のうち, 2 台を計算する用, 1 台を人間が操作する用にする.

OS: NixOS (nixos-unstable)
Languages:
　Rust (Cargo 1.97.1)
　TypeScript (Bun 1.3.13)

詳細

詳細は GitHub Pages にて公開しています.

https://yukkku.github.io/procon37-ube/docs/

23 BinaryCORE_A

秋田

佐々木 隆稀（4年） 佐藤 虹太（4年）
竹下 大樹（教員）

1.概要

評価関数を用いて効率よくうどんを集められるアルゴリズムを目指した。特に、巡回車と補給車の配置と距離の関係を意識した。

2.アルゴリズム

盤面が小さいときは、評価値を参考にして A*を用いて最もうどんを集められる経路を探索するようにした。盤面が大きいときは、まず A*を用いて移動可能なマスを探索し、その後ビームサーチを用いて回答の制限時間以内に、できるだけよい経路を探索するようにした。

また、巡回車と補給車の数と位置を決められることと、巡回車は補給車で燃料を補給しなければいけないことに注目した。小さい盤面の時は補給車を１台にし、補給車を中心とした円があると考え、その円からはみ出さないよう範囲で巡回車がうどんを回収することで効率よく燃料を補給できると考えた。この円の範囲内を１つのペアとみなすと、大きい盤面の時は補給車を２台にし、２ペアでそれぞれマップの半分を探索し、１日の後半で交代するという動きをするようなアルゴリズムにした。

3.GUI

現在どのように巡回車と補給車が動いているかを可視化するために、GUI を作成した。

4.開発環境

C++ Siv3D



図１　動作例

24 ヘキサンギ高専

都立(品川)

渡部 碧澄（3年） 田中 智之（4年）
菊谷 美咲（2年） 田中　覚（教員）

1. システム概要

本システムでは、サーバーや他プロセスとの通信、解探索、ビジュアライザーをそれぞれ独立したプロセスで管理している。独立したプロセスに分離することで、複数 solver の実行や複数 PC での分散化が容易になる。

2. エージェントの行動決定方法

初日に試合全体のエージェントの行動計画を生成し、日が変わり交通情報が更新されるたびに行動計画を修正していく。また、行動計画の生成にはルールベースの貪欲法や獲得種類数や累積種類数を評価関数に入れたビームサーチなどで決定する。

3. GUI

競技サーバーから取得した試合情報を元に現在の盤面や残り時間をリアルタイムに可視化する。Solver が生成した行動計画をアニメーションで再生して確認することができる。行動計画を GUI から提出できる。



図 2 開発中の GUI

5. 開発環境

OS: MacOS

言語: C++, Python

エディタ: VSCode

25 木製のうどん、Wooden Udon

沼津

藤田 康佑（4年） 皆神 賢吾（3年）
三村 治弘（3年） 鈴木 康人（教員）

1. 問題へのアプローチ

①当問題ではセル間の移動コストは非負整数かつ小さい数で抑えられることを利用して、Dial’s Algorithm を利用して経路探索を行う。特に経路の評価にも多用するため、可能な限り計算量を抑えた実装を施した。

②巡回車を効率的にスポット巡回させるために、①を利用して計算したコストを距離関数として k-medoids 法によってクラスタリングをし、それぞれのクラスタに巡回車を割り当てる。クラスタ内部の巡回方法は TSP を解く、ないし高速に近似して求める。

③焼きなまし法などのメタヒューリスティックによって巡回車の数や経路などのパラメータを探索し、効率的な手を見つけ出す。

2. 開発手法について

チームでの円滑な開発を目標とし、アーキテクチャの設計を工夫した。例えば盤面を受け取るクラスについて、依存関係を逆転し盤面の入力フォーマットと実装を切り離すことで、開発用の独自のフォーマットと本番でのフォーマットを同じように扱えるようにした。

3. 使用言語

C++20, C++23

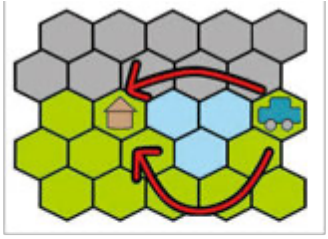
26 情緒ぶっかけうどん

奈良

加藤 大幹（2年） 福田 南（2年）
渡邊 怜亜（2年） 山口 賢一（教員）

1. 概要

この競技はスポットに到達するまでに、うどんの種類数、残りの燃料、地形などを考える必要がある。そのため、それらを基に盤面を評価し、ビームサーチで最適な解を見つけることを基本的な方針とする。また、盤面を分かりやすくするためビジュアライザを作成する。



ビームサーチで経路を探索するイメージ

2. アルゴリズム

巡回車は最も近いスポットへの最短経路を燃料の消費や渋滞、スポットの割り当てを考慮して枝分かれさせる。このとき、最短経路はダイクストラ法の要領で求め、他の経路は、最短経路の一部のマスを通らない経路を考えることで求める。補給車は巡回車の行動の目標をスポットから巡回車に変えたものにする。

3. 開発環境

環境：Windows 11、WSL
言語：C++、Python
エディタ：Visual Studio Code

27 釧路高専勇者部

釧路

益子 智至（3年） 白石 久人（3年）
藤村 優（3年） 天元 宏（教員）

1. 概要

今回の問題では，エージェントの燃料消費をあまり考慮せずに考えることが出来ると結論づけた．そうすることにより，巡回車がスポットを巡る問題とすることで，既存のアルゴリズムなどを用いやすくなった．

そのため，我々は複数のアルゴリズムによる手法を用意することで，リスクを分散した．

2.1 単純な巡回アルゴリズム

補給車と巡回車を 1:1 で割り当て，スポットを巡っていく．具体的には，一つの巡回車と補給車のみをメインで動かし，他の巡回車と補給車は別々のスポットに駐在するようにする．

2.2 巡回路生成アルゴリズム

最初に位置している場所や系列などが優秀なスポットをいくつか抜き出し，それらを一周するような経路を求める．その後，エージェントの初期位置から最も近い経路上のスポットへエージェントを移動させ巡回路に沿って巡回車を移動させる経路を求める．求めた経路を焼きなまし法を用い改善し，最後に経路を逆走するように補給車を動かす．

2.3 拠点移動アルゴリズム

スポットを 2 グループにクラスタリングし，その結果をもとにエージェントのクラスタリングも行う．それぞれのクラスタで拠点（R0 とする）を決め，その次の日の拠点位置（R1）を決める．R0 から R1 までの経路で近いスポットを追加などの操作を行うことで初期解を生成する．

その過程の最適化アルゴリズムとして焼きなまし法などのヒューリスティックアルゴリズムを用いる．

3. 開発環境

C++23, Golang, Python, CMake, VSCode, Windows11, macOS

28 Neural Nexus

阿南

米積 悠翔（4年） 後藤 貫（4年）
岡 康太（5年） 平山 基（教員）

1. 概要

本競技では、巡回車と補給車を協調させ、制限時間内にうどんを多く獲得することが求められる。本作品は、行動決定にマルチエージェント DQN、経路算出に条件付きダイクストラ法とビームサーチを組み合わせ、DQN が機能しない場合はルールベースへ切り替える。

2. アルゴリズム

2.1 DQN による行動決定

巡回車・補給車の燃料や敵の位置などから、「どのスポットに向かうか」「補給を優先するか」を決定する。報酬にはうどん獲得の期待値と燃料切れへのペナルティを反映する。

2.2 経路探索

DQN が決めた目標に対し、条件付きダイクストラ法とビームサーチで移動パスを算出する。DQN が使えない場合は、最寄りのうどん屋を選ぶなど簡易ルールで代替する。

2.3 協調と学習

巡回車のルート上に補給車が先回りし、効率よく燃料を補給できる仕組みとした。また、ルールベースの対戦相手と自己対戦学習を行い、学習の速度と精度を高めた。

3. ビジュアライザ

六角マップ上でログを再生するツールを用意した。

・車両の位置・燃料
・スポットの在庫
・ターン送りによる

3. 開発環境

OS：Windows, macOS, Linux
言語：C++, CUDA, Python
ライブラリ：Siv3D

29 いりこだしのうどんが好き

新居浜

久保 仁志（5年） 中家 海翔（5年）
金山 楓実（3年） 占部 弘治（教員）

1. 概要

本競技では、1ターンでの移動回数が多い都合上、一度の移動に対して計算を行うと膨大な量の計算が必要になる。スポットでのうどんの獲得が重要であることも考慮して現在地からどのスポットへ移動するかに焦点を当て処理を行うようにした。

2. アルゴリズムについて

2.1 スポットの評価

基本的に本競技での評価基準に沿って未獲得の系列であるか、今日未獲得の系列であるか、うどんをより多く獲得できるかの順でスポットの評価を行う。盤面の大きさやスポットの数に応じてその次に向かうスポットも考慮に入れるなど、マップに合わせた戦略をとる。

また、目的地の重複を避けるため巡回車ごとに担当する領域を割り当て、行動を行う。補給車は当日中に新たな系列へ到達できうる巡回車を優先して燃料の補給を行う。

2.2 経路選択

各スポットまでの経路をダイクストラ法で探索し、到達に必要なステップ数が最小で消費燃料がより少ない経路を選択する。

2.3 巡回車と補給車のバランス

様々なマップでのシミュレーションを行い、平均スコアの高かった割合を本戦では採用する。

3 開発環境

OS：Microsoft Windows 11 Home（64ビット）

Language：Python 3.13.14

30 高専低学年の部

富山 (射水)

京角 こと（2年） 黒川 千尋（2年）
宮崎 哉月（1年） 秋口 俊輔（教員）

1. 概要

本競技では、各エージェントへの訪問先の割当て、スポットの訪問順序、セル間の経路を同時に決定する必要がある。これらをすべて探索すると組合せが膨大になるため、スポットと初期位置を重要地点とし、重要地点間の経路を辺とするグラフに圧縮する。

2. アルゴリズム

2.1 経路探索

地形によって移動時間と燃料消費が異なるため、最短時間の経路が最も燃料効率のよい経路になるとは限らない。そこで、時間重視や燃料重視など、条件を変えながら探索する。得られた経路から、追加の移動時間と燃料消費が少ないものだけを残す。

2.2 順路計画

補給車と補給拠点の複数の組合せをそれぞれ比較する。スポットを追加するときは、試合全体でまだ獲得していない系列を優先する。その中から、制限時間と燃料上限を満たす効率の良いものを選択する。

3. ビジュアライザー

ビジュアライザーでは、地形、スポットの位置と系列、エージェントの位置・残り燃料などを表示する。行動をステップ単位で確認することで、経路の重複、燃料不足、補給のタイミングなどを発見し、アルゴリズムの改善に利用する。

4. 開発環境

Solver ： C++

Tester ： Rust

Visualizer ： HTML, CSS, JavaScript

31 うどんで海の水、全部抜く。

松江

田中　庵（5年）　中里 太一（3年）
林 千聖怜（3年）　橋本　剛（教員）

1. 概要

１日のエージェント全体の行動は解空間が大きいため、ソルバーにはメタヒューリスティックスアルゴリズムを採用する。最終日以外では翌日以降の道路状況が不確実であり、長期的に安定した行動が必要となる。また、道が多いマップやスポットの少ないマップなど、多様なマップが存在し、マップによりゲーム性が異なると考えられる。そのため、多種多様かつ不確実な未来を考慮した解法を用意する。

2. アルゴリズム

2.1 エージェント選択について

自チームの補給車の台数が０～２の場合を候補として、それぞれ簡易ソルバーでシミュレーションをして評価を行い、最も優れたエージェント選択を選定する。

2.2 1日の行動列について

１つ目の解法構造の概要は、最終日まで簡易シミュレーションして仮定した道路状況の下、評価を実スコアとした焼なまし法を用いて最終日までの行動の最適化を行い、1日目の行動を取得し、採用することである。２つ目の解法構造の概要は、1日の行動の候補集合をビームサーチにより構築し、実スコア以外も評価として取り入れ、最も優れた候補を選定することである。これらの解法構造の中で、並列化やパラメータチューニングなど様々な工夫や手法を取り入れる。

3. 開発環境

言語：C++23, Python3.12.3, HTML5, CSS3, JavaScript
エディタ：Visual Studio Code
OS：Windows11, WSL(Windows Subsystem for Linux)2
生成AIツール：Codex, Claude Code

32 伊勢うどん

鳥羽商船

金城 空之介（2年）　奥村 瑛斗（2年）
田中 力也（2年）　廣瀬　誠（教員）

1. 概要

・本競技では、巡回車の燃料制限、渋滞による必要ステップ数の変動、スポットまでの必要ステップ数と燃料数など、評価に必要な項目が多くなっている。
・本システムでは、ダイクストラ法により完全グラフを生成し、ルート選定には、獲得うどん種類数、移動ステップ数、燃料切れリスクを統合した評価関数を採用しました。
・フィールド全体を探索すると計算量が膨大になるため、ビームサーチを用いて数手先を見据えた高スコア経路をリアルタイムに探索する。

2. 実装上の工夫

・1ステップでもマップ外、池の侵入、燃料不足があると回答自体が却下となるため、送信前に確認できるように安全装置を搭載する。
・視覚的な情報で試合を進行していくため、ビジュアライザを実装した。



図1：ビジュアライザ

4. 開発環境

Turbowarp Desktop

33 徹夜最適化班

都城
津曲　涼（3年）　田中 陸哉（2年）
平野 太一（2年）　丸田　要（教員）

1. はじめに

本チームは「ヘキサうどん」において、限られた思考時間内で効率的にスポットを巡回し、得点を最大化するアルゴリズムを構築した。

2. 1探索系アルゴリズム

移動経路は A*探索で算出し、六角形マップを三次元キューブ座標で管理することで高速化した。巡回順序はビームサーチで決定し、短時間でも効率的なルート選択を可能にした。

3. 戦略と図の説明

初日は道路が順調なため遠方スポットへ移動し、2 日目以降は渋滞を避けて平地・山地を利用した裏道ルートを選択した。 図 1 は自作ビジュアライザーの出力であり、地形、スポット在庫、巡回車・補給車の位置、燃料状態を可視化し、戦略の妥当性を検証するために使用した。

4. まとめ

A*探索・ビームサーチ・渋滞回避戦略・Z スコア評価を組み合わせることで、短時間でも安定して高得点を狙えるアルゴリズムを実現した。図 1 は本システムの動作を可視化したビジュアライザーの出力である。



図 1　ヘキサうどん競技マップの可視化画面

5. 開発環境

使用言語：C#

エディタ：Visual Studio Code

34 ナフィノート

群馬
山縣 蒼人（5年）　生方 來夢（5年）
渡邉 俊哉（教員）

1. 概要

ソルバー、ビジュアライザー、問題のジェネレーター、サーバー送受信用のインターフェースなどを作成した。

2. ソルバー

獲得したうどんの系列数が最も重要であるため、独自に設計した評価関数を用いて、解答の最適化を実行している。

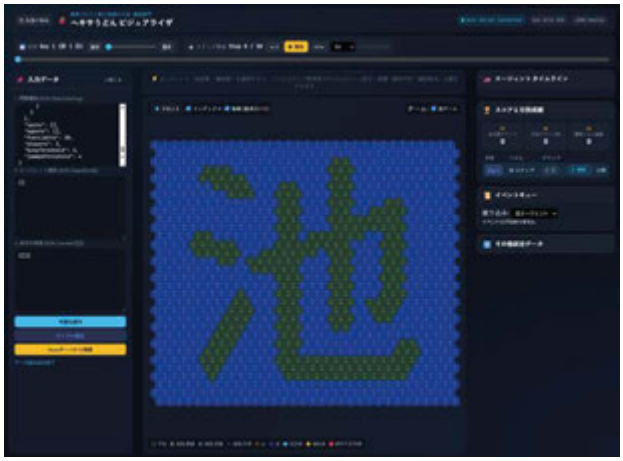
3. ビジュアライザー

3.1 概要

TypeScript を用いた Web アプリケーションとして、本問題のビジュアライザを作成した。

2.2 機能

各地形を色分けしたマップやスポット、エージェントをうどんストック数や燃料の残量などを含めて表示できる。また、エージェントの移動の軌跡やうどんの獲得、燃料補給といったイベントビューワーなど、さまざまな角度から視覚化が可能である。



4 ジェネレーター

問題の特性上、良質なテストケースを作成することが難しいため、テストケースを簡単に作成できるジェネレーターを作成した。

35 Unfixed

長岡
千葉 幹太（3年） 津輕 健太（3年）
高梨 優杜（2年） 竹部 啓輔（教員）

1. 概要

今回の競技部門では、我々は貪欲解と機械学習で得た解を導き、より良い回答を提出する戦略を取る。

2. アルゴリズム

まず、エージェントを約半分ずつ巡回車と補給車に分け、両者からなるグループを作成する（奇数で余るエージェントは巡回車とする）。次に、日々の道路状況に応じてDijkstra 法で最短距離を計算し、グループ内の全エージェントの「最大移動距離が最小」となる地点を集合場所に選定して同期させる。

集合後は、未取得系列を強く優先する評価関数を用いた貪欲法にランダム性を加え、500 回試行して得られた最適ルートを巡回する。この際、巡回車と補給車が同ルートを走行して常に給油を行うことで、燃料の残量を考慮する必要をなくした。

最後に、実際の行動をシミュレーションして状態を更新し、翌日は新たな道路状況に基づき再度ルートを構築する。

3. ビジュアライザ

開発・研究での調整を容易にするため、ビジュアライザを作成した。上のバーを進めることでステップを進められるようになっている。

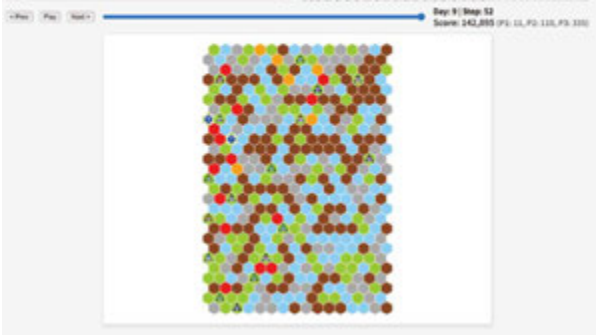


図 1 ビジュアライザ

36 うどんをいっぱい食べ隊

長野
野村 岳歩（5年） 南 大地（3年）
笠谷 春太朗（2年） 吉原 一成（教員）

1. システム概要

今回の競技では、巡回車と補給車を連携させ、制限時間内により多くのうどんを獲得することを目指す。スポット間の移動コストやうどんの種類を基に巡回領域を分離し、近いスポット群の中で効率的に巡回経路を探索する。

2. アルゴリズム

まず、ダイクストラ法によってスポット間の最短移動コストを事前計算し、その情報とうどんの系列をもとにスポットをクラスタリングする。実際の経路探索には D* Lite を用い、交通状態が変化した際に、必要な部分だけを再計算して更新する。さらに、未獲得のうどんの種類、移動コスト、在庫数、残燃料などを評価して巡回経路を決定し、補給車についても巡回車の燃料状況を予測して合流地点を計画する。

3. ビジュアライザ

無効な移動が回答に含まれると回答全体がリジェクトされるため、送信前に解を厳密に検証し、無効な解の提出を防ぐ。また、送受信処理とビジュアライザの描画は 1 台の PC で行い、経路探索は別の計算用 PC で行う。



図 1：ビジュアライザ

4. 開発環境

使用言語：Rust, C++, Python

フレームワーク：Nannou

37 丸投げせぇへん?

弓削商船
岡田 慈英（専攻科２年）
藤田 章良（１年）　木原　　陸（１年）
榎本 浩義（教員）

1. はじめに

今回の競技では、達成可能なうどんの種類数の最大値を達成することを、その次に一日当たりに取得できるうどんの種類数を最大化させることを目的とする。

2. アルゴリズム

クラスタリングアルゴリズムを用いて、複数のスポットをいくつかのクラスタに分け、巡回車に担当のエリアを持たせ、そのエリア内で種類数の最大化を行う。クラスタの数はエージェントの数より少なくすることで、補給車用に余りを用意し、各クラスタを巡回するように行動させる。

3. GUI

アルゴリズムの考察や、エージェントが実際にどのような動きをしているかを確認するため、GUI を用意する。



図 1　開発中の GUI

4. 開発環境

使用言語：Java

OS：Windows11

38 どうしま商船

大島商船
米谷 倫太郎（３年）　河本 拳輔（３年）
國本 晴紀（３年）　北風 裕教（教員）

1. アルゴリズム

盤面は最大 32×32 で、全ての行動の組合せを調べる全探索は現実的ではない。そこで、最短経路を求めるダイクストラ法、近い有望な店舗を選ぶ貪欲法、複数の候補を残して比較するビーム型探索を組み合わせる。候補は step 単位のシミュレーションで合法性と結果を確認し、最良の計画を提出する。原則としてエージェント 0 を補給車、残りを巡回車とし、盤面の大きさに応じて探索量を自動調整する。

2. 評価関数

店舗選択と候補比較には、次の評価式を用いる。

店舗評価（小さいほど優先）

＝移動 step＋2×燃料消費＋各種ペナルティ－30×新ブランド－3×在庫数

移動時間、燃料消費、不利な条件には加点して選ばれにくくし、新ブランドと在庫には減点して選ばれやすくする。

内部評価点（大きいほど優先）

＝500×ブランド数＋100×うどん数＋残燃料－4×移動回数－80×在庫切れ訪問

これらは、大会ルール上の優先順位を数値化し、ランダム盤面などのテスト結果を確認しながら係数を調整した「経験則に基づく重み付き評価（ヒューリスティック）」である。

内部評価点は公式得点ではなく、候補を比較するための補助値である。実際には、まず計画の合法性、次に取得ブランド種類数、うどん数などを順番に比較し、内部評価点は最後の補助判断として使用する。

3. その他の工夫

通常巡回、店舗分担、補給車との合流、緊急補給、複数の巡回車への連続補給など、複数の計画候補を生成する。また、探索期限を設定し、時間切れになった未完成候補は破棄して、検証済みの候補を保持する。validation、step simulation、submit check を通過した計画だけを採用し、異常時には全待機計画を安全策として使用する。ログと盤面表示により、経路、燃料、うどんの取得状況も確認できる。

4. 開発環境

OS：Windows 11

開発環境：Visual Studio 2022、Visual Studio Code

言語：C++17

主なライブラリ・機能：nlohmann/json、WinHTTP

39 そんなことよりおうどんたべたい

和歌山
山本 理雄眞（3年） 山吹 研蔵（3年）
小出 英範（教員）

1. システムの概要

本システムは六角形の盤面上で複数の車両を動かし、うどんを集める行動計画を生成します。獲得玉数だけでなくより多くの系列を集めることを重視し、限られた燃料と行動時間をチーム全体で有効に使います。中核部分と問題生成器を Rust で実装し、Python で評価を自動化して改良を重ねています。

2. 経路探索の工夫

地形や道路状況から移動に必要な時間と燃料を求め、各車両が訪問できるスポットを調べます。その日にまだ獲得していない系列やスポットの在庫を考慮して、チーム内で訪問先を割り当てます。到達できる系列の少ない巡回車を先に計画する候補も比較し、車両ごとの到達範囲の違いを生かします。

さらに、巡回車のみ 3 台の構成ではビーム探索を用い、複数の経路候補を残しながら組合せを探します。当日終了時の位置と残燃料から翌日の到達しやすさも見積もります。補給車を含む構成では、同じ位置からの同行や、巡回車の終点での待合せを候補に含めます。

3. 確実に回答する仕組み

行動計画は提出前にローカルシミュレータで検証します。燃料消費、移動、うどんの獲得、補給をステップごとに再現し、燃料不足や時間超過を検出します。探索が時間切れになっても回答を残せるよう、先に待機だけの計画を提出し、その後に検証済みの行動計画へ更新します。

開発では公式資料の具体例に基づく試験と、多様な盤面での繰返し評価を行います。経路や補給の様子を確認しながら、得点と安定性の両立を目指します。

40 寿司よりうどん

近畿大
藤岡 慶春（4年） 瀧野 寿也（4年）
山中 和希（4年） 坂東 将光（教員）

1. 解法

本問題では全体最適を求めるための探索空間が大きいため、全体の方針を決める上位戦略と実際の移動経路を効率的に決定する下位制御に分割して考える。

2. システム概要

本システムは、バックエンドとフロントエンドを明確に分離することで、GUI を介した直感的な状況把握や競技中の安全な手動介入を可能にしている。

3. システム構成

システムの全体構成を図 1 に示す。

3.1 フロントエンド

Browser を通じて操作を受け取り、盤面のリアルタイム表示およびバックエンドとの通信を行う。

3.2 バックエンド

回答側サーバーからエントリー処理を経て、回答出力を行う。内部の GameEngine および Strategy が最善手を算出する。

3.3 ホストサーバー

通信モジュールを介し、大会側の回答受理サーバーへデータを送信する。

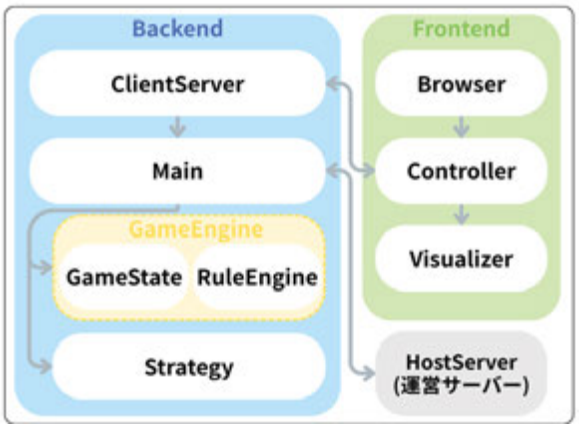


図 1 システム構成図

41 新入部員募集中

仙台 (名取)

村上　陸（4年）　黒澤 徹真（4年）
北島 宏之（教員）

1. はじめに

今回の競技では，複数のエージェントを操作し，マップ上に配置されたスポットからできるだけ多くのうどんを獲得することが求められる．

2. アルゴリズムについて

2.1 アルゴリズムの流れ

我々のチームでは，まずマップ上のスポットとエージェントの位置関係を調べ，各スポットを効率よく巡回するための経路を探索する．巡回車はスポットに到着することでうどんを獲得できるため，できるだけ多くのスポットを訪れることを基本の流れとした．

2.2 スポットまでの経路探索

経路探索にはダイクストラ法を使用し，各セルの移動コストを重みとして扱う．スポットを選択する際は，うどんの種類，巡回に必要なステップ数，燃料消費量，現在地からの距離などを考慮する．今回はエージェントの動きが燃料によって制限されることから，少ない移動で新しい種類のうどんを獲得できるスポットを優先することで，限られた時間の中で多くの種類を獲得することを目指した．

3. 工夫

燃料を補給する補給車をマップの中央付近に固定することで補給車の不要な移動を減らし，数を必要最低限にすることができ，余ったリソースを巡回車にまわすことができると考えている．

4. 開発環境と使用言語

開発環境：Visual Studio Code
開発言語:C++

42 コーナーで差をつける春光台モビリティ部

旭川

川上 晃弥（4年）　越後　奨（4年）
嶋田 鉄兵（教員）

1. アルゴリズムの工夫

全マス間の最短経路を BFS で事前計算し、系列網羅を最優先にスポットの優先度を算出する。各ターンの行動候補はビームサーチで数手先まで展開し、評価値の高い行動系列に絞り込むことで、局所最適に陥らず探索の精度と速度を両立させる。道路の混雑・渋滞状況や他チームの位置から翌日の交通量を予測し、渋滞回避と燃料切れ防止を両立させるルートを選択する。巡回車・補給車の役割分担と合流タイミングも 1 ステップ先読みして計画する。

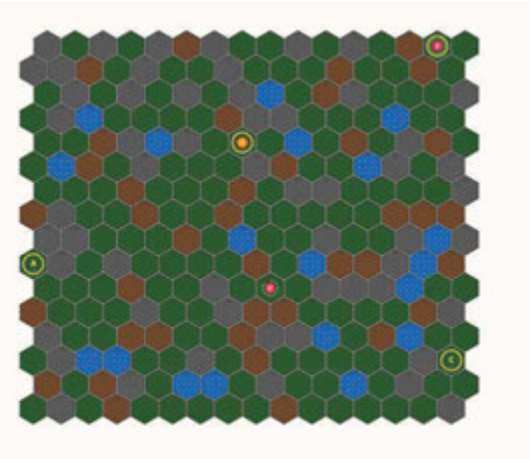


図 1　自作ビジュアライザによるエージェント巡回の可視化

2. 開発体制

Python で仕様検証とアルゴリズム試作を進め、pytest による 104 件のテストで品質を担保する。実行速度確保のため C++へ移植し、下図の自作リプレイビジュアライザで巡回車・補給車の動きを確認しながら改善を重ねている。

43 ゴリゴリズム

熊本 (熊本)
永松 日月（5年） 茶薗 優斗（3年）
野中 一真（3年） 神﨑 雄一郎（教員）

1. 概要

勝敗は種類数を最優先とする辞書式で決まる。盤面に良い評価値を与えることが難しいため、1日分の全エージェント計画を1手とする探索を採用した。

2. アルゴリズムについて

2.1 行動について

巡回車1台が1日に取りうる行動列は、移動回数をmとして最大7^m通りである。8台ではその8乗となり、$m = 40$でも$7^{8m} = 7^{320} \simeq 10^{270}$に達する。最大10日続くため全探索は困難である。

2.2 Chokudai Search

数日先の計画探索には、ビームサーチの亜種であるChokudai Searchを用いた。深さごとに候補を保持し、各深さから有望な状態を順次展開することで、限られた時間内で複数日先の計画を探索する。

3. ビジュアライザ

図1に示すビジュアライザを作成した。リプレイを読み込み、以下をステップ単位で表示することができる。

盤面と各エージェントの位置・燃料

スポットの在庫と道路の状態

両チームの種類数・杯数の推移

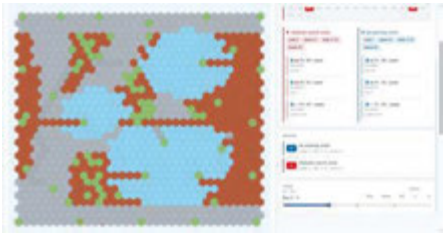


図 1 ビジュアライザのスクリーンショット

4. 開発環境

C++、TypeScript、Python

React、PyTorch

44 骨付鳥取県

米子
池口 寛大（5年） 秋田 功太郎（4年）
田中 拓弥（5年） 川戸 聡也（教員）

1. 戦略概要

本チームでは、相手の行動の不確実性を考慮し、相手との衝突をできるだけ避けることに重きを置いた戦略を採用する。まず、フィールド上の移動可能なマスを頂点、移動を辺とするグラフとして表現する。次に、「貪欲に最短経路を通る」「できるだけコストの小さい道を選ぶ」などの相手の行動規則をもとに同じ局面から多数回の試行を行い、各マス・各時刻における道の通過頻度を危険度として蓄積する。

2. 危険度を用いた経路探索

得られた危険度は、相手が実際に通過したマスだけでなく、その周辺や前後の時刻にも半分の重みで加算する。これにより、予測誤差に対する頑健性を確保した危険度分布を作成する。これを経路探索の重みとし、距離だけでなく相手との遭遇確率も考慮した経路を選択する。

3. 高速化

グラフの隣接関係にはビット列を利用し、危険度も整数値を用いることで、浮動小数と比べて計算時間を短縮する。1回のシミュレーションを軽量化することで、制限時間内に可能な限り多くの試行を行う。そして、試行回数の増加によって相手の動きの予測精度を高め、より質の高い適切な自機の行動決定に活かす。

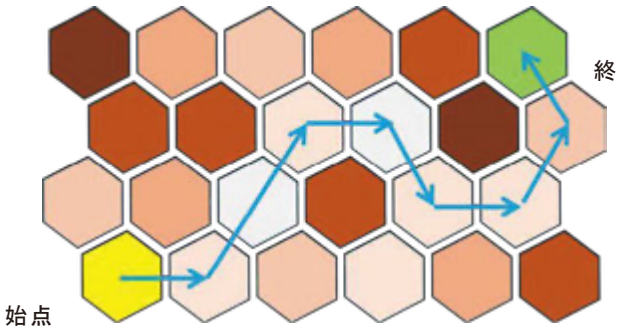


図 危険度分布を考慮した経路探索のイメージ

45 うどんよりラーメン派

鶴岡

伊藤 吉宗（4年） 坂本 蒼太（4年）
柴田 蒼（3年） 金 帝演（教員）

1. 概要

与えられた盤面上で複数の車両を動かすために、各車両の移動経路や担当する範囲、行動を決定する必要がある。そこで、経路探索、担当範囲の分割、行動選択に対して、それぞれに適したアルゴリズムを採用した。

2. アルゴリズム

2.1 経路探索

各車両の移動経路を決定するため、Dijkstra 法を用いて経路探索を行う。また、六角形の盤面に対応するため Cube 座標を用い、2 地点間の距離を計算する。

2.2 担当範囲の分割

複数の車両に担当する範囲を割り当てるため、クラスタリングを用いて盤面を複数の領域に分割する。これにより、各車両が担当する領域を決定する。

2.3 行動選択

各状態に対して評価値を計算し、その値をもとに次の行動を選択する。探索する候補を絞るためにビームサーチを用い、評価値の高い候補を残しながら探索を行う。

3. ビジュアライザ

車両の動きや盤面の状態を確認するため、ビジュアライザを作成した。主に以下の情報を表示する。

- 盤面の状態
- 各車両の位置
- 日数ごとのステップ数



4. 開発環境

言語 C/C++ ライブラリ SFML

46 手打ち（詐欺）

神山

相木 絆煌（4年） 丹那 伊織（4年）
若宮 小芭音（3年） 山地 駿徹（教員）

1. はじめに

私たちは、複数のエージェントをマップ上に配置し、燃料上限やステップ数といった制約の中で、より多くのうどんを獲得するための探索アルゴリズムを開発しています。特に、単純に最短経路を求めるのではなく、「どのエージェントがどのスポッのタイミングで補給するか」まで含めて最適化することを目指しています。トを担当するか」「どの順番で巡回するか」「ど

2. 使用するアルゴリズムについて

2.1 エージェントの配置と巡回

まず、与えられたエージェント数、各エージェントの燃料上限、マップ上のスポット配置をもとに、DFS（深さ優先探索）を用いてエージェントの配置と担当範囲を決定します。この段階で担当範囲が偏らないように分担することで、エージェント同士の移動の重複を減らし、燃料消費を抑えます。その後、それぞれのエージェントについて、燃料上限を制約条件として到達可能なスポット系列をDFSで列挙し、全スポットを巡回できる実行可能な初期解を高速に構築します。

2.2 制限時間内にうどんを多く獲得する工夫

初期解を得た後は、残りの計算時間を使って山登り法による改善を行います。具体的には、スポットの担当、訪問順序、燃料補給のタイミングなどを変更し、評価値が向上した場合に解を更新します。すべての訪問順を総当たりで探索すると組み合わせが階乗的に増加するため、現在の解に近い候補のみを探索することで計算量を抑えています。また、山登り法では局所最適解に陥る可能性があるため、複数の初期解から探索を行うことで、より良い解を見つけやすくする工夫も取り入れています。

2.3 評価関数について

解の評価には、競技の勝敗条件に合わせた評価関数を使用します。具体的には、まず試合全体で獲得したうどんの種類数を比較し、次に日ごとの種類数の累積、最後にうどんの総獲得玉数を比較します。

2.4 ビジュアライザ

アルゴリズムそのものだけでなく、その挙動を検証するためのWebビジュアライザも開発しています。TypeScriptとViteを用いて、日ごとの各エージェントの動きや状態、スコア、エラーなどを可視化し、どの部分に無駄な移動があるのか、どの制約で問題が起きているのかを確認できるようにします。これにより、アルゴリズムの挙動を視覚的に分析しながら、改善のサイクルをより高速に回せる開発環境を作っています。

47 食ベルせぇるすまん

香川（高松）
後藤 勇輝（4年） 東原 遙希（4年）
作花 優二（4年） 高谷 秀明（教員）

1. 概要

今回の競技で可能な操作は六角形のマス上で車両を動かすだけだが、日をまたいで盤面や燃料の状態が複雑に変化するため、データ構造を工夫し複数の盤面を効率的に保持することが大切である。

2. アルゴリズム

今回は学習させたAIモデルで手を生成する。

まず時間をかけた探索で質の高い手を大量に作り、それを教師データとしてモデルに模倣学習させる。学習したモデルで次の教師データをさらに安く作り、再び学習させることを繰り返して精度を高めている。

これにより、探索なら数十分かかる質の手を数十秒で決定でき、1日ごとの回答時間制限の中でも良い手を返すことができる。

3. GUI

図1のように盤面を可視化し、アルゴリズムに不具合が発生した場合に迅速な対応ができる。また、車両の移動経路に線を引き、位置を分かりやすく表示している。



図1　盤面の表示画面

4. 開発環境

OS: Ubuntu

使用言語: C++、Python

IDE: Visual Studio Code

48 久留米ニヨン人

久留米
日永田 幸多（3年） 堀川 海凪（2年）
針山 真次（1年） 原田 裕二郎（教員）

1. 解法

1.1 アルゴリズム

巡回車ごとに各スポットへの最短距離を計算し、どのスポットにどの巡回車を向かわせるかを、ビームサーチを用いて探索する。各巡回車における燃料残量についての問題は、巡回車の移動を事前に確定させ、燃料が尽きるステップの直前に補給車を横切らせることで解決する。

1.2 うどんの入手数を最大化する工夫

全種類のうどんを獲得したのち、さらに獲得量を増やすため、巡回車ごとに残りのステップ数と燃料残量を考慮し、到達可能なスポットを探索する。その後、各巡回車に向かうスポットをそれぞれ指定し、各巡回車の経路の最適化させる。

2. プログラムの性能検証

異なるアルゴリズムのプログラムを作成し、それを仮想の相手として競わせることで、想定される相手チームに対する性能を測る。

3 ビジュアライザ

HTML, CSS, JavaScript, Python を使用して、簡易な web アプリでビジュアライザを作成した(図 1)。



図 1

49 Udontu Noble Noodle

豊田

吉野 弘晟（3年）
ターラーンガン タナバディー（5年）
山口 到真（3年） 三浦 哲平（教員）

1. アルゴリズム

1.1 エージェントの割り振り

補給車と巡回車をバランスよく割り振る。巡回車と補給車を1:1とすれば、巡回車は燃料切れを起こすことなく行動し続けられるが、効率は良くない。補給車を回収車よりも少なく設定し、回収に動けるエージェントを多めにとることで、より多くのうどんを回収することを狙う。

1.2 エージェントの行動計画

エージェントの移動を全パターン考えるのには膨大な計算が必要になり、与えられた時間を超過してしまう。そこで、回収車の移動はスポットに向かうものと、燃料を補給するものだけに限定し、無駄な動きの探索に時間を割かないようにした。1日の行動計画に与えられた計算時間を最大まで使って行動の改善をする。

燃料や移動ステップの制約を正確に計算するため、エージェントのセル、燃料、現在のステップ数をまとめて1つの状態として管理し、自然に状態遷移を考えられるようにした。

2. ビジュアライザ

ビジュアライザは可視化機能だけを持ち、軽量な html ファイル単体で動作する。配布や整備の簡単さと使いやすさを重視した。

3. その他

チーム内でより良いアイデアを模索するため、簡易的な対戦機能を実装し、チームメイトのモチベーション向上と解法のブラッシュアップを図った。

本番のマップがどのように生成されるかは問題文から把握できなかったため、手元で条件を満たすランダムケースを生成してベンチマークを行った。

50 ATSUagain

福井

桑野 利成（4年） 秦 啓大（4年）
藤田 稜治（4年） 斉藤 徹（教員）

1. 概要

本競技では同じ盤面上で複数人が対戦する。よって、複数のアルゴリズムを組み合わせることで各スポットへの最善経路を求めるだけでなく、簡易的な相手の行動予測も含めた最も効率的な経路で、スポットの巡回を行う。

2. アルゴリズム

2.1 エージェントの割り当て

エージェントがそれぞれ違う位置からスタートするということを考慮し、考えられる全ての振り分けを試すことで最も良いと考えられる組み合わせを求める。

2.2 相手の行動分析

相手のエージェントの位置などから翌日までにどう動き、どこに移動するかを予測して行動決定に用いる。

2.3 行動決定

各回収車が担当するスポットはそれぞれが算出した期待利得によって調整を行い、全体的な効率を高める。そしてダイクストラ法などの様々なアルゴリズムを用いて最短経路を基準とした最適な経路を決定する。また、ダイクストラ法におけるコストには試合の勝敗条件をもとにした評価関数や、相手の行動分析によって考えられる翌日の渋滞状況を考慮している。

3. 開発環境

OS : Windows11

言語 : C++

IDE : Visual Studio Code

51 鹿児島高専

鹿児島

新中 大貴（3年） 野妻 奏太（3年）
中岡 小虎（2年） 武田 和大（教員）

1. 経路探索

マップの各セルを頂点、移動可能な方向を辺とするグラフとして扱う。地形ごとに設定された移動時間と燃料消費量を辺のコストとし、ダイクストラ法によってスポット間の経路を求める。経路は、移動時間を優先する最短経路と、燃料消費量を優先する省燃料経路の2種類を生成し、巡回車の状態に応じて使い分ける。

2. 巡回ルートの生成

各巡回車の現在地を始点とし、ビームサーチによって1日に巡る複数のスポットと、その順番を探索する。ルートは、新たに獲得できるうどんの種類数、スポットの希少性、獲得玉数、移動時間、燃料消費量をもとに評価する。

スポットの希少性は、その日の時間と燃料で到達できる巡回車数から求める。到達できる巡回車が少ないスポットを優先することで、取得機会を失いやすいうどんを回収する。また、各巡回車の候補ルートを組み合わせ、スポットの在庫や巡回車同士の重複を考慮し、巡回車全体で獲得数が多くなる行動を選択する。

3. 補給車の先回り

決定した巡回ルートを共通の時間軸へ展開し、各ステップの巡回車の位置と残り燃料を計算する。燃料不足が予測された場合は、燃料切れより前に巡回車と補給車が同じセルで合流できる地点を探索する。

複数の合流地点から、補給車の待機時間が少なく、補給後も巡回車が長く行動できる地点を選択し、補給車を経路途中へ先回りさせる。補給できない巡回ルートは不採用とし、別の候補ルートへ切り替える。

4. 開発環境

言語：Rust

UI：Tauri

52 気炎万丈（ねっけつかいわい）

鈴鹿

市川 敬士（5年） 小笠原 朋（5年）
中村 春琉（5年） 桑野 一成（教員）

1. スポットまでの経路を探索するアルゴリズム

ダイクストラ法を用いて，各スポットまでの最短経路を探索する。競技では平地・山地・道路など地形ごとに移動時間や燃料消費量が異なり，道路については混雑状況によって移動コストが変化する。そのため，各セル間の移動コストを重みとして扱い，巡回車が目的地へ最も効率よく到達できる経路を算出する。また，単純な距離の短さだけでなく，燃料補給地点との位置関係や，将来的なスポット巡回効率も考慮することで，中長期的に有利となる移動計画を実現する。これにより，限られたステップ数の中でも多くのスポットを巡回し，うどんの種類数および獲得数の最大化を目指す。

2. 制限時間内にうどんを多く獲得する方法

制限時間内により多くのうどんを獲得するため，ビームサーチを用いた行動探索アルゴリズムを採用する。競技では，エージェント数やスポット数が多く，すべての行動パターンを全探索することは現実的ではない。そのため，各状態から生成される行動候補の中から，評価値の高い候補のみを一定数保持しながら探索を進めることで，計算量を抑えつつ高得点が期待できる行動を効率的に探索する。評価関数には，獲得可能なうどんの種類数，移動効率，燃料残量，将来的な巡回可能性など複数の要素を組み込み，短期的な利益だけでなく，将来的な得点期待値も考慮した探索を行う。

3. 開発環境

C++, Rust, Go, Python, TypeScript

Docker, VSCode, Github

53 ミートソースパスタ

サレジオ

大島 和喜（3年） 谷岡 里桜（2年）
樋口 湊（1年） 永岡 淳一（教員）

1. 概要

巡回車と補給車という 2 つのエージェントの最短ルートを導出するために、Dijkstra 法と A*アルゴリズムを用いて最短経路を効率的に求めることを目指す。最適なアルゴリズムを採用して、最小のステップ数で移動を行うことを目指す。

2. 探索アルゴリズム

2.1 巡回車の最適ルートの導出

巡回車の最適ルートを探索するアルゴリズムとして、Dijkstra 法を用いる。本競技では一つの巡回車に対して目的地であるうどんを獲得するためのスポットが複数存在しているため最短経路を求められる Dijkstra 法を用いて探索を行う。Dijkstra 法は優先度付きキュー(min-heap)を用いて実装することで処理時間の短縮を図る。

2.2 補給車の最適ルートの導出

補給車の最適なルートを導出すると燃料の切れるタイミングが予測できるため先回りして燃料を補給する方式を採用する。補給する地点は一点に定まるため A*を用いて探索を行う。

3. 開発環境

使用言語 :C++

開発環境 :Visual Studio Code

54 巡回セールス麺

大阪公大

水谷 航太（4年） 藤本 悠生（2年）
岩口 桃羽（5年） 窪田 哲也（教員）

1. はじめに

今回のお題は、「有限行動の巡回車 + 無限行動の給油車 + 時間経過で価値が積み上がる」という構造で、巡回セールスマン問題やリソース制約付き計画に関連があると考えられる。また、評価関数が系列数、日ごとの累計系列数、累計うどん数の辞書順なので、「単純にうどん数最大化」ではなく、かなり戦略が分かれる。

2. アルゴリズム

巡回車の経路の組み合わせだけでもスポット数の階乗ある。その上、給油車と燃料などの制約もあるため、全探索などは難しい。そこで、我々は確率的貪欲法をベースに解法プログラムを構築した。特に工夫した点は、評価関数である。確率的貪欲法では、貪欲法に乱数を導入し、何度も解を生成させて評価関数をかける。この時、試合時の評価を再現した本番スコアと最後の盤面を評価する盤面スコアにそれぞれ重みをつけた評価関数を作成した。これによって次の日にもつ繋げられる関数設計を実現した。

3. ビジュアライザ

試合本番に向けて、評価関数のスコアや確率的貪欲法に利用するパラメータなどを様々用意して実際の試合を完璧に想定したシミュレーションを行いそれぞれのモデルのパフォーマンスを可視化する。また、ランダムサーチやベイズ最適化などを用いてパラメータチューニングを行う。

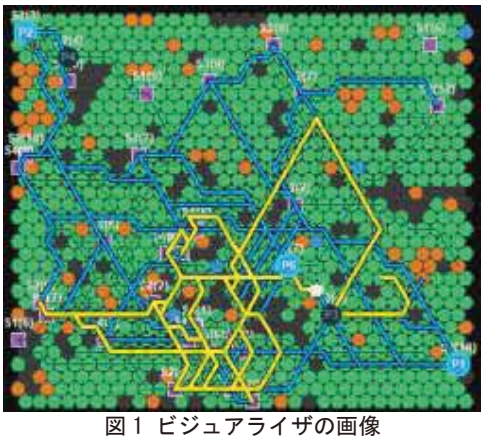


図 1 ビジュアライザの画像

4. 開発環境

言語：C++

ライブラリ：

55 人力全探索

小山

森　悠喜（4年）　八性 亮仁（3年）
北嶋 心一（1年）　小林 康浩（教員）

1. はじめに

勝敗条件の優先度はうどんの種類数が最も高いため、多くの種類のうどんを集めることを優先する。そのため堅実にうどんを収集するプログラムを作成することにした。

2. プログラムの概要

ダイクストラ法によって各エージェントから最も近いスポットまでのステップと燃料を考慮し計算する。

スポットまで移動しうどんを獲得した後、その日に獲得したうどんの種類が被らないようにスポットを選択し再計算する。

エージェントについて巡回車と補給車を半々にわけ、巡回車と補給車１つずつでペアを作り、ペアの巡回車と補給車の間隔ができるだけ狭くなるように設定する。補給車は巡回車に追従させるようにする。

3. ビジュアライザ

作成したプログラムがどう動くのかを視覚的に分かりやすくかつ、サーバー側の処理を再現するために図１のようなビジュアライザを作成した。



図１　ビジュアライザの実行例

１ステップごとの移動、マップの色分け、獲得したうどんの表示の機能だけでなく、実際に JSON で入出力する機能、パラメータをファイルの値を書き換えることによってランダムもしくは適当な値にできる機能、エージェントの動きを矢印として残す機能を盛り込んだ。

4. 開発環境

使用言語: C++
開発環境: VSCode, Visual Studio (siv3d), Windows11

56 味噌らぁめんの加護を授かりし者共

神戸市立

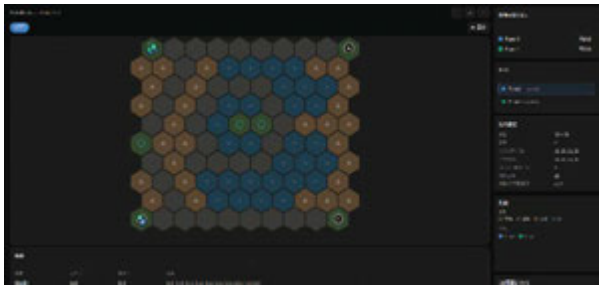
布谷　陸（2年）　砂川 拓穂（1年）
朝倉 義裕（教員）

1.はじめに

このチームでは、まず公式ルールを再現したシミュレーターを実装し、複数の戦略を同一条件で比較・検証できる環境を構築しました。基本戦略として、各スポットまでの所要ステップ数とうどんの価値から巡回経路を決定するアルゴリズムを実装しています。さらに、日々変化する道路状況を考慮するため、複数日先までシミュレーションして行動を比較する探索型アルゴリズムの実装を進めています。また、制限時間内に確実に回答できるよう、探索規模を調整可能な構成としています。加えて、日付や渋滞状況、燃料、獲得したうどんなどを確認できるビジュアライザも実装しました。

2.アルゴリズム

まず初めに、公式ルールをそのまま再現したシミュレーターを実装し、その上で複数の戦略を同じ条件で比較・検証できるようにしました。基本となる戦略として、各スポットまでの所要ステップ数と獲得できるうどんの価値を評価し、効率よく巡回するアルゴリズムを実装しました。しかし道路の状態は数日にわたって変化するた



め、その日だけを見た判断では不十分だと考え、現在は複数の行動候補を生成し、数日先までシミュレーションした結果を公式の勝敗基準に従って比較する探索型のアルゴリズムの実装を進めています。また、各日の回答時間内に確実に提出できるよう、探索の規模を調整できる構成にしました。

3. ビジュアライザ

現在のマップ状況を描画し、日付、渋滞状況、エージェントの残り燃料、獲得したうどんの種類、種類数の累積、玉数の総和を表示させました。

4. 開発環境

使用言語: Python
エディタ: Visual Studio Code

57 OSU

新モンゴル

Iveel Enkhtur
Tuguldur Dashrentsen
Tsetsenbileg Altangerel
Soronzonbold Nachin（教員）

HEXUDON Agent: A Parallel Algorithm Portfolio Scored by an Exact Simulator

Reg. No. 30081 / OSU / NMCT / Mongolia

1. Approach

For our own score each day is deterministic: opponents cannot touch our stock, and road conditions are known when the day begins. We attack it with four search algorithms in parallel on sixteen CPU cores.

2. Exact Simulator as the Sole Arbiter

Our foundation reproduces the official rules exactly: the reflection/action phase split, source-cell movement cost, cell occupancy while in transit, and stock contention by agent order. It is validated against the worked example in the official Q&A, and every candidate we generate is scored by it, so our objective cannot drift from the server's.

3. Reliability, Not Average Quality

Scoring is lexicographic: distinct udon types outrank the daily type total, which outranks servings — one missed type outweighs every serving in a match. We encode this exactly in 64-bit integers, then optimise for reliability: fifteen diversified lanes, four of which never exchange solutions, turn a per-lane success probability p into $1-(1-p)^k$.

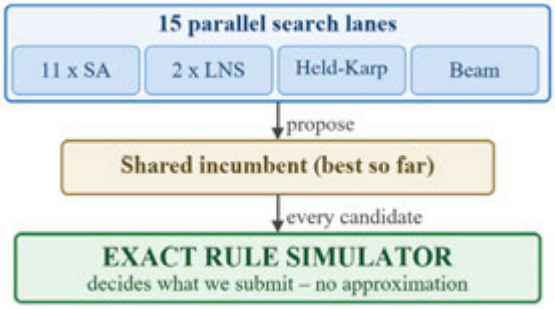


Fig. 1 Four algorithm families propose; only the exact simulator decides.

4. Cross-Day Planning

Fuel carries across days, so greedy per-day play strands vehicles later. Each plan is credited with the types its end state could still reach tomorrow, bounded below one real type-day so it never overrides exact scoring. On fuel-scarce maps this removed our variance entirely.

5. Results

On the contest machine: 7.2× search throughput over one thread, no data races under ThreadSanitizer, and a valid plan within 2.3 s even in a five-second window.

58 EastUniversity

モンゴル科技大

Munkhbagana Munkhtur
Bilguuntuguldur Purev-Ochir
Khuder Altangerel（教員）

1. Introduction

The HEXUDON field is modeled as a dynamic weighted hexagonal graph, with each cell as a node and edges connecting valid neighboring cells per the official layout rule (even rows shifted right, non-torus map).

2. Implementation Method

2.1 Pathfinding to Udon Spots

For each patrol car, the system generates multiple reachable candidate routes using Dijkstra/A*, with an edge cost dynamically computed from terrain movement cost, fuel consumption, road condition, predicted congestion risk, and remaining daySteps.

Rather than choosing the nearest spot, each candidate is evaluated with an expected-reward model.

success_probability is estimated from reachability, remaining fuel, refueling support, stock availability, and the chance another patrol car reaches the spot first.

2.2 Strategy for Maximizing Udon Collected

The solver performs adaptive daily re-planning with multi-agent coordination. Patrol cars are assigned to high-value spots while refueling cars act as mobile support units, in several stages.

A local simulator reproduces the official processing order (fuel consumption, movement, udon acquisition, refueling, traffic update) to catch timing errors before submission.

Figure 1. Sample route visualization.



競技部門

59 Perfect Dominance

ハノイ国家大
Nguyen Tung Duong
Hoang Quoc Viet
Ma Thị Châu（教員）

1. Introduction

The problem provides a hexagonal grid map where multiple teams compete in a multi-day match. Each team controls patrol and refueling vehicles and must coordinate their movements to collect as many udon as possible while managing vehicle constraints and limited resources.

2. Algorithm

We use the Adaptive Large Neighborhood Search (ALNS) algorithm, consisting of two main components: Space Search and Evaluation.

2.1 Space Search

We first generate an initial plan using a greedy strategy, where each patrol vehicle moves to the nearest available spot until the end of the day. Starting from this plan, we repeatedly destroy part of the solution and repair it to generate a new plan. If the new plan has a better evaluation score, it is accepted. This process continues until the daily time limit is nearly reached. For refueling vehicles, we similarly use a greedy strategy to visit spots where patrol vehicles can meet and refuel. The same destroy-and-repair mechanism is then applied to improve their routes.

2.2 Evaluation

The evaluation function considers both current performance and future potential. For the current day, the three objectives of the problem are prioritized hierarchically, with Objective 1 receiving higher priority than Objective 2, and Objective 2 higher than Objective 3. In addition, we consider future potential, such as the remaining fuel of patrol vehicles at the end of the day and whether their final positions provide good and advantageous starting points for the following day.